

1. Decision Control Statements:

Write a Python program to determine whether a given number is prime or composite using if, elif, and else statements. The program should prompt the user to input a number and display whether the number is prime, composite, or neither (e.g., if the number is 1).

```
def is_prime(n):  
    """Returns True if n is a prime number, False otherwise."""  
    if n <= 1:  
        return False  
    for i in range(2, int(n**0.5) + 1):  
        if n % i == 0:  
            return False  
    return True  
  
num = int(input("Enter a number: "))  
  
if num == 1:  
    print(f"{num} is neither prime nor composite.")  
elif is_prime(num):  
    print(f"{num} is a prime number.")  
else:  
    print(f"{num} is a composite number.")
```

Sample Output:

```
Enter a number: 7  
7 is a prime number.  
  
Enter a number: 12  
12 is a composite number.  
  
Enter a number: 1  
1 is neither prime nor composite.
```

2. Looping and Nested Loops:

Write a Python program that prints the Floyd's Triangle using nested for loops. The program should take an integer n as input and print the first n rows of the triangle. The triangle should look like this for n = 5:

```
1  
2 3
```

```
4 5 6
7 8 9 10
11 12 13 14 15
```

```
def print_floyds_triangle(n):
    """Prints Floyd's Triangle with n rows."""
    num = 1
    for i in range(1, n + 1):
        for j in range(1, i + 1):
            print(num, end=" ")
            num += 1
        print() # Move to the next line after each row

# Get input from the user
n = int(input("Enter the number of rows for Floyd's Triangle: "))

# Print the triangle
print_floyds_triangle(n)
```

Sample Output:

```
Enter the number of rows for Floyd's Triangle: 5
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

1. List, Dictionary, and Set Comprehensions:

Write a Python program using list comprehension, dictionary comprehension, and set comprehension to:

- Create a list of all the even numbers between 1 and 50.
- Create a dictionary where keys are numbers from 1 to 5 and values are their squares.
- Create a set of unique characters from the string "programming".

```
# List Comprehension
even_numbers = [num for num in range(1, 51) if num % 2 == 0]
print("Even numbers:", even_numbers)

# Dictionary Comprehension
squares_dict = {num: num**2 for num in range(1, 6)}
print("Squares dictionary:", squares_dict)

# Set Comprehension
unique_chars = {char for char in "programming"}
print("Unique characters:", unique_chars)
```

Output:

Even numbers: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48, 50]

Squares dictionary: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

Unique characters: {'a', 'g', 'm', 'o', 'p', 'r', 'i', 'n'}

2. Arrays: 1D and 2D Arrays in Python:

Write a Python program that:

- Creates a 1D array using the array module and initializes it with integers.
- Creates a 2D array using the numpy module and fills it with random numbers between 1 and 100.
- Accesses the element at position (1, 2) in the 2D array and prints it.

```
import array as arr
import numpy as np

# Create a 1D array using the array module
my_array = arr.array('i', [1, 2, 3, 4, 5])
print("1D Array:", my_array)

# Create a 2D array using the numpy module
two_d_array = np.random.randint(1, 101, size=(3, 3))
print("2D Array:\n", two_d_array)

# Access the element at position (1, 2) in the 2D array
element = two_d_array[1, 2]
print("Element at (1, 2):", element)
```

Output:

```
1D Array: array('i', [1, 2, 3, 4, 5])
2D Array:
[[ 7 42 83]
 [98 54 56]
 [74 45 41]]
Element at (1, 2): 56
```

1. Functions and Recursion:

Write a Python program that defines a recursive function to compute the factorial of a given number n. Additionally, implement a non-recursive version of the factorial calculation in a separate function.

```
def is_prime(n):
```

```
    if n <= 1:
```

```
        return False
```

```
    for i in range(2, int(n**0.5) + 1):
```

```
        if n % i == 0:
```

```
            return False
```

```
    return True
```

```
num = int(input("Enter a number: "))
```

```
if num == 1:
```

```
    print(f'{num} is neither prime nor composite.')

```

```
elif is_prime(num):
```

```
    print(f'{num} is a prime number.')

```

```
else:
```

```
    print(f'{num} is a composite number.')

```

```
def print_floyds_triangle(n):
```

```
    num = 1
```

```
    for i in range(1, n + 1):
```

```
        for j in range(1, i + 1):
```

```
            print(num, end=" ")
```

```
            num += 1
```

```
        print()
```

```
n = int(input("Enter the number of rows for Floyd's Triangle: "))
```

```
print_floyds_triangle(n)
```

```
even_numbers = [num for num in range(1, 51) if num % 2 == 0]
```

```
print("Even numbers:", even_numbers)
```

```
squares_dict = {num: num**2 for num in range(1, 6)}
```

```
print("Squares dictionary:", squares_dict)
```

```
unique_chars = {char for char in "programming"}
```

```
print("Unique characters:", unique_chars)
```

```
import array as arr
```

```
import numpy as np
```

```
my_array = arr.array('i', [1, 2, 3, 4, 5])
```

```
print("1D Array:", my_array)
```

```
two_d_array = np.random.randint(1, 101, size=(3, 3))
```

```
print("2D Array:\n", two_d_array)
```

```
element = two_d_array[1, 2]
```

```
print("Element at (1, 2):", element)
```

```
def factorial_recursive(n):
```

```
    if n == 0:
```

```
        return 1
```

```
    else:
```

```
        return n * factorial_recursive(n - 1)
```

```
def factorial_iterative(n):
```

```
    result = 1
```

```
    for i in range(1, n + 1):
```

```
        result *= i
```

```
    return result
```

```
num = int(input("Enter a number: "))
```

```
recursive_factorial = factorial_recursive(num)
```

```
iterative_factorial = factorial_iterative(num)
```

```
print(f"Factorial of {num} (recursive): {recursive_factorial}")
```

```
print(f"Factorial of {num} (iterative): {iterative_factorial}")
```

Output:

Decision Control Statements:

Enter a number: 7

7 is a prime number.

Enter a number: 12

12 is a composite number.

Enter a number: 1

1 is neither prime nor composite.

Looping and Nested Loops:

Enter the number of rows for Floyd's Triangle: 5

1

2 3

4 5 6

7 8 9 10

11 12 13 14 15

List, Dictionary, and Set Comprehensions:

Even numbers: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48, 50]

Squares dictionary: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

Unique characters: {'a', 'g', 'm', 'o', 'p', 'r', 'i', 'n'}

Arrays: 1D and 2D Arrays in Python:

1D Array: `array('i', [1, 2, 3, 4, 5])`

2D Array:

`[[74 15 63]`

`[98 62 40]`

`[13 53 45]]`

Element at (1, 2): 40

Functions and Recursion:

Enter a number: 5

Factorial of 5 (recursive): 120

Factorial of 5 (iterative): 120

2. Lambda Functions:

Write a Python program that uses a lambda function to:

- **Take a list of integers as input from the user.**
- **Use the lambda function to filter out the even numbers from the list.**
- **Print the list of even numbers.**

```
numbers = list(map(int, input("Enter a list of integers (separated by spaces): ").split()))
```

```
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
```

```
print("Even numbers:", even_numbers)
```

Output:

Enter a list of integers (separated by spaces): 1 2 3 4 5 6 7 8 9 10

Even numbers: [2, 4, 6, 8, 10]

1. Searching Techniques: Linear and Binary Search:

Write two separate functions:

- A linear search function to find an element in an unsorted list.
- A binary search function to find an element in a sorted list. For both functions, the program should display the index of the element if found, or a message saying the element was not found.

```
def linear_search(arr, x):
```

```
    for i in range(len(arr)):
```

```
        if arr[i] == x:
```

```
            return i
```

```
    return -1
```

```
def binary_search(arr, x):
```

```
    low = 0
```

```
    high = len(arr) - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if arr[mid] < x:
```

```
            low = mid + 1
```

```
        elif arr[mid] > x:
```

```
            high = mid - 1
```

```
        else:
```

```
            return mid
```

```
    return -1
```

```
arr = [2, 3, 4, 10, 40]
```

```
x = 10
```

```
result = linear_search(arr, x)
```

```
if result != -1:
```

```
    print("Element is present at index", str(result))
```



```

else:

    print("Element is not present in array")

arr.sort()

result = binary_search(arr, x)

if result != -1:

    print("Element is present at index", str(result))

else:

    print("Element is not present in array")

```

Output:

Element is present at index 3

Element is present at index 3

2. Sorting Techniques: Selection Sort and Bubble Sort:

Write a Python program that implements both Selection Sort and Bubble Sort algorithms to sort a given list of integers in ascending order. The program should:

- Take a list of integers as input from the user.
- Sort the list using both sorting techniques.
- Print the sorted list for both sorting algorithms.

```

def selection_sort(arr):

    for i in range(len(arr)):

        min_idx = i

        for j in range(i+1, len(arr)):

            if arr[j] < arr[min_idx]:

                min_idx = j

        arr[i], arr[min_idx] = arr[min_idx], arr[i]

def bubble_sort(arr):

    n = len(arr)

```

```
for i in range(n):  
    for j in range(0, n-i-1):  
        if arr[j] > arr[j+1]:  
            arr[j], arr[j+1] = arr[j+1], arr[j]  
  
# Get input from the user  
numbers = list(map(int, input("Enter a list of integers (separated by spaces): ").split()))  
  
# Create copies of the original list  
selection_sort_list = numbers.copy()  
bubble_sort_list = numbers.copy()  
  
# Sort the lists using the respective algorithms  
selection_sort(selection_sort_list)  
bubble_sort(bubble_sort_list)  
  
# Print the sorted lists  
print("Selection Sort:", selection_sort_list)  
print("Bubble Sort:", bubble_sort_list)
```

Output:

Enter a list of integers (separated by spaces): 64 34 25 12 22 11 90

Selection Sort: [11, 12, 22, 25, 34, 64, 90]

Bubble Sort: [11, 12, 22, 25, 34, 64, 90]